

Workshop 5: Coordination Distributed Systems

Dario Pomasqui

School of Mathematical and Computational Sciences

Yachay Tech University

Professor: Prof. Francisco Hidrobo

San Miguel de Urucuquí, Imbabura, Ecuador

GitHub Repository: [https:](https://github.com/adrianasofiaespinoza/Sistemas-Distribuidos/tree/main/Workshop5)

[//github.com/adrianasofiaespinoza/Sistemas-Distribuidos/tree/main/Workshop5](https://github.com/adrianasofiaespinoza/Sistemas-Distribuidos/tree/main/Workshop5)

September 17, 2026

Abstract

This report presents the implementation and analysis of four coordination mechanisms for distributed systems: (1) centralized time synchronization via a UTC server using ZeroMQ, (2) decentralized global time synchronization inspired by the Berkeley algorithm, (3) vector clocks for causal event ordering, and (4) mutual exclusion through both a central coordinator with a FIFO queue and a distributed token ring. All implementations use Python with ZMQ for inter-process communication and threading for parallel execution. Results and discussions are provided for Parts 2, 3, and 4 as requested.

1 Introduction

Coordination in distributed systems involves managing the interactions and synchronization of multiple independent processes that do not share a global clock or memory. This workshop addresses four fundamental coordination problems:

- **Centralized time synchronization** using a UTC server (Part 1).
- **Decentralized time synchronization** without a central authority (Part 2).
- **Causal event ordering** through vector clocks (Part 3).
- **Mutual exclusion** via centralized and distributed algorithms (Part 4).

All programs were implemented in Python 3 using ZeroMQ (ZMQ) for message passing and the `threading` module for concurrency. The source code is available in the GitHub repository linked above.

2 Part 1: Coordination through UTC Server

2.1 Objective

Test a client-server program where a centralized UTC server provides the current time to multiple clients. The original code was enhanced with input validation and exception handling.

2.2 Architecture

The system follows the ZMQ REQ/REP pattern. The server binds a REP socket on port 5000 and waits for incoming requests. Each client creates a REQ socket, sends a time request, and receives the UTC timestamp.

2.3 Improvements over Original Code

Bug fix in client threading. The original client code called the function immediately instead of passing it as a reference:

Listing 1: Original (buggy)

```
1 # The parentheses cause immediate execution in the main thread
2 c.append(threading.Thread(target=utc_time_client(), args=()))
```

Listing 2: Fixed version

```
1 # Pass the function reference without parentheses
2 t = threading.Thread(target=utc_time_client, args=())
```

Timeout and exception handling. The improved client sets a receive timeout to avoid blocking indefinitely if the server is unreachable:

Listing 3: Timeout configuration

```
1 socket.setsockopt(zmq.RCVTIMEO, 5000) # 5-second timeout
2 socket.setsockopt(zmq.LINGER, 0)      # Don't block on close
```

Graceful server shutdown. The server uses `socket.poll()` with a timeout instead of a blocking `recv()`, allowing it to respond to `Ctrl+C`:

Listing 4: Non-blocking server loop

```
1 try:
2     while True:
3         if socket.poll(timeout=1000):
4             message = socket.recv()
5             utc_time = time.strftime('%Y-%m-%d %H:%M:%S', time.gmtime())
6             socket.send_string(utc_time)
7 except KeyboardInterrupt:
8     print("Servidor detenido")
9 finally:
10    socket.close()
```

2.4 Discussion

The REQ/REP pattern enforces a strict request-reply sequence, making it suitable for time queries. However, this centralized approach introduces a single point of failure: if the UTC server crashes, no client can synchronize. Part 2 addresses this limitation with a decentralized approach.

3 Part 2: Global Time without a UTC Server

3.1 Objective

Simulate decentralized time synchronization where each peer node maintains its own clock, exchanges timestamps with other peers, and adjusts its local time based on the average. Random drift is introduced every k cycles to model real-world clock skew.

3.2 Algorithm

The approach is inspired by the **Berkeley algorithm**. Each peer maintains a simulated clock:

$$T_{local} = T_{system} + \text{offset}$$

During each synchronization round:

1. A peer collects timestamps from all other peers via ZMQ REQ/REP.
2. It computes the average: $T_{avg} = \frac{1}{n} \sum_{i=1}^n T_i$
3. It adjusts its offset: $\text{offset}_{new} = T_{avg} - T_{system}$
4. Every $k = 2$ rounds, random drift $\in [-5, +5]$ s is added.

3.3 Key Implementation Details

Each peer runs a **server thread** that responds to time requests, and a **main loop** that periodically synchronizes:

Listing 5: Clock adjustment after collecting peer times

```

1 if len(times) > 1:
2     promedio = sum(times) / len(times)
3     with self.lock:
4         old = self.clock_offset
5         self.clock_offset = promedio - time.time()

```

The drift injection simulates real clock inaccuracy:

Listing 6: Periodic drift injection

```
1 if (r + 1) % DRIFT_EVERY_K == 0:
2     drift = random.uniform(-MAX_DRIFT, MAX_DRIFT)
3     with self.lock:
4         self.clock_offset += drift
```

3.4 Configuration

Parameter	Value	Description
NUM_PEERS	3	Number of peer nodes
BASE_PORT	6000	Base port (6000, 6001, 6002)
SYNC_INTERVAL	3s	Time between rounds
DRIFT_EVERY_K	2	Add drift every 2 cycles
MAX_DRIFT	5s	Maximum random drift
NUM_ROUNDS	10	Total synchronization rounds

Table 1: Peer node configuration

3.5 Results and Discussion

Each peer starts with a random offset in $[-3, +3]$ s. After each synchronization round, the offsets converge as each peer adjusts toward the group average. However, every 2 rounds, the injected drift disrupts convergence, and the system must re-synchronize.

The algorithm is resilient because it does not depend on a single authority. If one peer is unreachable, its time is excluded from the average, and the remaining peers still converge. The main limitation is that the average can be biased if an outlier peer reports a significantly skewed time.

```
C:\Users\User\Desktop\Bos\Sistemas-Distribuidos\Sistemas-Distribuidos\Workshop\Actividad2_Global_Tiempo
PS C:\Users\User\Desktop\Bos\Sistemas-Distribuidos\Sistemas-Distribuidos\Workshop\Actividad2_Global_Tiempo> python peer_node.py 0
[Peer 0] Escuchando en puerto 6000
[Peer 0] Ronda 1 - Tiempo antes: 18:34:42 (offset: +1.399s)
[Peer 0] Promedio de 3 peers: Ajuste: -2.352s
[Peer 0] Tiempo despues: 18:34:45 (offset: -0.952s)
[Peer 0] Ronda 2 - Tiempo antes: 18:34:48 (offset: -0.952s)
[Peer 0] Promedio de 3 peers: Ajuste: +0.106s
[Peer 0] Tiempo despues: 18:34:50 (offset: -0.846s)
[Peer 0] Drift aplicado: -2.113s
[Peer 0] Ronda 3 - Tiempo antes: 18:34:51 (offset: -0.977s)
[Peer 0] Promedio de 3 peers: Ajuste: +0.487s
[Peer 0] Tiempo despues: 18:34:54 (offset: -2.489s)
[Peer 0] Ronda 4 - Tiempo antes: 18:34:57 (offset: -2.489s)
[Peer 0] Promedio de 3 peers: Ajuste: -2.400s
[Peer 0] Tiempo despues: 18:35:00 (offset: -2.761s)
[Peer 0] Drift aplicado: +2.297s
[Peer 0] Ronda 5 - Tiempo antes: 18:35:03 (offset: -0.400s)
[Peer 0] Promedio de 3 peers: Ajuste: -2.925s
[Peer 0] Tiempo despues: 18:35:06 (offset: -2.359s)
[Peer 0] Ronda 6 - Tiempo antes: 18:35:09 (offset: -2.359s)
[Peer 0] Promedio de 3 peers: Ajuste: -0.212s
[Peer 0] Tiempo despues: 18:35:12 (offset: -2.571s)
[Peer 0] Drift aplicado: +4.062s
[Peer 0] Ronda 7 - Tiempo antes: 18:35:15 (offset: +1.089s)
```

(a) Execution output (Part 1)

```
C:\Users\User\Desktop\Bos\Sistemas-Distribuidos\Sistemas-Distribuidos\Workshop\Actividad2_Global_Tiempo
PS C:\Users\User\Desktop\Bos\Sistemas-Distribuidos\Sistemas-Distribuidos\Workshop\Actividad2_Global_Tiempo> python peer_node.py 2
[Peer 2] Ronda 8 - Tiempo antes: 18:35:18 (offset: +1.361s)
[Peer 2] Promedio de 3 peers: Ajuste: -0.627s
[Peer 2] Tiempo despues: 18:35:21 (offset: +1.989s)
[Peer 2] Drift aplicado: -1.521s
[Peer 2] Ronda 9 - Tiempo antes: 18:35:24 (offset: +1.989s)
[Peer 2] Promedio de 3 peers: Ajuste: -2.238s
[Peer 2] Tiempo despues: 18:35:27 (offset: +4.227s)
[Peer 2] Drift aplicado: -1.077s
[Peer 2] Ronda 10 - Tiempo antes: 18:35:30 (offset: +4.227s)
[Peer 2] Promedio de 3 peers: Ajuste: -4.972s
[Peer 2] Tiempo despues: 18:35:33 (offset: -0.745s)
[Peer 2] Drift aplicado: +1.672s
```

(b) Execution output (Part 2)

Figure 1: Output logs of the decentralized global time synchronization.

4 Part 3: Vector Clocks

4.1 Objective

Implement vector clocks to track causal relationships between events in a distributed system, enabling the distinction between causally ordered and concurrent events.

4.2 Vector Clock Rules

Given n processes, each process P_i maintains a vector $VC_i[0..n-1]$:

- **Internal event:** $VC_i[i] \leftarrow VC_i[i] + 1$
- **Send event:** $VC_i[i] \leftarrow VC_i[i] + 1$; attach VC_i to the message.
- **Receive event** (from P_j with timestamp TS):

$$\forall k : VC_i[k] \leftarrow \max(VC_i[k], TS[k]), \quad \text{then } VC_i[i] \leftarrow VC_i[i] + 1$$

Causality: $a \rightarrow b$ iff $VC(a)[k] \leq VC(b)[k]$ for all k and $VC(a) \neq VC(b)$.

4.3 Key Implementation Details

The `VectorClock` class encapsulates the update rules. The `on_receive` method merges the incoming clock with the local one:

Listing 7: Vector clock merge on receive

```
1 def on_receive(self, other_clock):
2     with self.lock:
3         for i in range(self.n):
4             self.clock[i] = max(self.clock[i], other_clock[i])
5             self.clock[self.pid] += 1
```

Messages are serialized as JSON and transmitted via ZMQ PUSH/PULL sockets. Each process binds a PULL socket on port $7100 + \text{pid}$ and sends through temporary PUSH sockets:

Listing 8: Sending a message with piggybacked vector clock

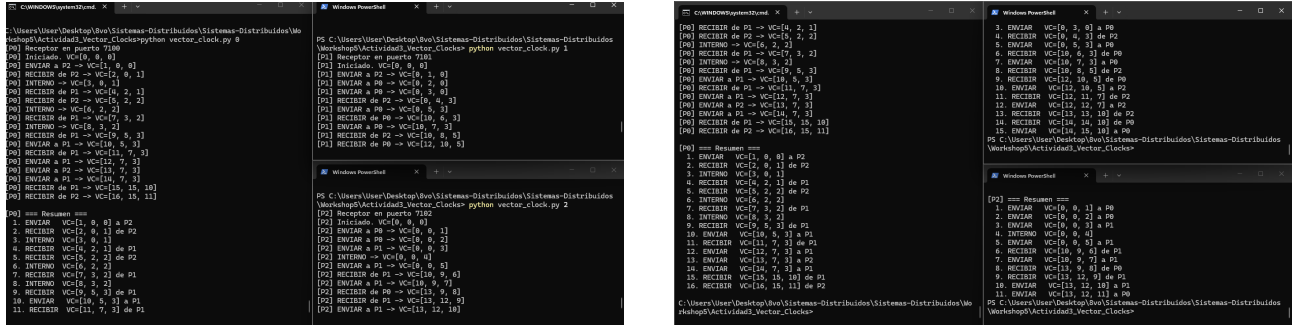
```
1 clock = self.vc.on_send()
2 msg = json.dumps({"sender": self.pid, "clock": clock})
3 socket.send_string(msg)
```

Events are selected randomly at each step (biased toward `send` events to generate inter-process communication). At termination, each process prints a summary of its event log.

4.4 Results and Discussion

With 3 processes running 8 events each, the vector clocks correctly reflect causal dependencies. For example, if P_0 sends a message to P_1 at $VC_0 = [2, 0, 0]$, and P_1 receives it, P_1 's clock becomes $[\max(VC_1[0], 2), VC_1[1] + 1, \max(VC_1[2], 0)]$, correctly encoding that P_1 has knowledge of P_0 's history up to that point.

Unlike Lamport timestamps, vector clocks can determine **concurrency**: if $VC(a) \not\leq VC(b)$ and $VC(b) \not\leq VC(a)$, events a and b are concurrent. The trade-off is $O(n)$ overhead per message, which is acceptable for small n but becomes costly at scale.



(a) Execution output (Part 1)

(b) Execution output (Part 2)

Figure 2: Output logs of the Vector Clocks activity, showing causal ordering.

5 Part 4: Mutual Exclusion

5.1 Objective

Implement mutual exclusion guaranteeing that at most one process accesses a shared resource at any time, using two approaches: a centralized coordinator and a distributed token ring.

5.2 Approach 1: Central Resource Management Server

5.2.1 Protocol

The coordinator uses a ZMQ ROUTER socket; clients use DEALER sockets with unique identity strings. The message exchange follows this sequence:

1. Client $\rightarrow$ Coordinator: LOCK_REQUEST
2. Coordinator $\rightarrow$ Client: LOCK_GRANTED (if free) or LOCK_QUEUED (if busy)
3. Client performs work in the critical section.
4. Client $\rightarrow$ Coordinator: RELEASE
5. Coordinator grants access to next client in FIFO queue (if any).

5.2.2 Key Implementation Details

The coordinator maintains a deque as a FIFO queue and tracks the current holder:

Listing 9: Coordinator lock management

```
1 if msg == "LOCK_REQUEST":
2     if self.holder is None:
3         self.holder = client_id
4         socket.send_multipart([client_id, b"", b"LOCK_GRANTED"])
5     else:
6         self.queue.append(client_id)
7         socket.send_multipart([client_id, b"", b"LOCK_QUEUED"])
```

Upon release, the coordinator dequeues the next waiting client:

Listing 10: Coordinator release handling

```
1 elif msg == "RELEASE":
2     socket.send_multipart([client_id, b"", b"RELEASE_ACK"])
3     if client_id == self.holder:
4         self.holder = None
5         if self.queue:
6             nxt = self.queue.popleft()
7             self.holder = nxt
8             socket.send_multipart([nxt, b"", b"LOCK_GRANTED"])
```

5.3 Approach 2: Distributed Token Ring

5.3.1 Protocol

Processes form a logical ring $P_0 \rightarrow P_1 \rightarrow P_2 \rightarrow P_0$. A single token circulates; only the token holder may enter the critical section. Each process uses a PULL socket to receive the token and a PUSH socket to forward it.

5.3.2 Key Implementation Details

Process 0 injects the initial token into the ring:

Listing 11: Token creation by Process 0

```
1 if self.pid == 0:
2     time.sleep(2)
3     tmp = self.context.socket(zmq.PUSH)
4     tmp.setsockopt(zmq.LINGER, 1000)
5     tmp.connect(f"tcp://{HOSTNAME}:{self.my_port}")
6     time.sleep(0.5)
7     tmp.send_string("TOKEN")
8     tmp.close()
```

When the token arrives, the process decides probabilistically whether to enter the critical section:

Listing 12: Token handling and critical section

```

1 token = pull.recv_string()
2 if token == "TOKEN":
3     quiere_cs = random.random() < CS_PROB
4     if quiere_cs and self.cs_count < NUM_ROUNDS:
5         self.cs_count += 1
6         time.sleep(random.uniform(0.5, CS_TIME))
7     push.send_string("TOKEN") # Forward token

```

5.4 Comparison

Criterion	Central Server	Token Ring
Coordination	Centralized	Distributed
Single point of failure	Yes	No
Fairness	FIFO guaranteed	Ring order
Messages per CS entry	3	$O(n)$ worst case
Latency (if free)	Low	Variable
Token loss risk	N/A	Yes (deadlock)

Table 2: Comparison of mutual exclusion approaches

5.5 Results and Discussion

The **central server** guarantees strict FIFO fairness: the first requester is the first served. The ZMQ ROUTER/DEALER pattern allows the coordinator to identify clients by their socket identity, enabling targeted responses. The drawback is the single point of failure.

The **token ring** distributes coordination across all processes, eliminating the single point of failure. However, it introduces two issues: (1) wasted bandwidth when the token circulates among processes that do not need the resource, and (2) risk of deadlock if the token is lost (e.g., a process crashes while holding it). The probabilistic model ($\text{CS_PROB} = 0.6$) simulates realistic workloads where not every process requires the resource each time.

Both approaches guarantee **safety** (at most one process in the critical section) and **liveness** (every request is eventually granted, assuming no failures).

```

\Workshop5\Actividad4_Mutual_Exclusion> "C
PS C:\Users\User\Desktop\8vo\Sistemas-Distribuidos\Sistemas-Distribuidos
\Workshop5\Actividad4_Mutual_Exclusion> "C
PS C:\Users\User\Desktop\8vo\Sistemas-Distribuidos\Sistemas-Distribuidos
\Workshop5\Actividad4_Mutual_Exclusion> python central_server.py client
0
[Cliente-0] Acceso concedido - entrando a seccion critica
[Cliente-0] Saliendo de seccion critica
[Cliente-0] Acceso concedido - entrando a seccion critica
[Cliente-0] Saliendo de seccion critica
[Cliente-0] En cola...
[Cliente-0] Acceso concedido - entrando a seccion critica
[Cliente-0] Saliendo de seccion critica
PS C:\Users\User\Desktop\8vo\Sistemas-Distribuidos\Sistemas-Distribuidos
\Workshop5\Actividad4_Mutual_Exclusion>

LOCK_GRANTED a Cliente-0
LOCK_QUEUED Cliente-1 (cola: 1)
LOCK_QUEUED Cliente-2 (cola: 2)
LOCK_GRANTED a Cliente-1
LOCK_QUEUED Cliente-0 (cola: 2)
LOCK_GRANTED a Cliente-2
LOCK_QUEUED Cliente-0 (cola: 2)
LOCK_GRANTED a Cliente-0
LOCK_QUEUED Cliente-2 (cola: 2)
LOCK_GRANTED a Cliente-1
LOCK_QUEUED Cliente-2 (cola: 1)
LOCK_GRANTED a Cliente-1
Recurso Libre
LOCK_GRANTED a Cliente-2
Recurso Libre

\Workshop5> cd .\Actividad4_Mutual_Exclusion\
PS C:\Users\User\Desktop\8vo\Sistemas-Distribuidos\Sistemas-Distribuidos
\Workshop5\Actividad4_Mutual_Exclusion> python central_server.py client
1
[Cliente-1] En cola...
[Cliente-1] Acceso concedido - entrando a seccion critica
[Cliente-1] Saliendo de seccion critica
[Cliente-1] En cola...
[Cliente-1] Acceso concedido - entrando a seccion critica
[Cliente-1] Saliendo de seccion critica
[Cliente-1] En cola...
[Cliente-1] Acceso concedido - entrando a seccion critica
[Cliente-1] Saliendo de seccion critica
PS C:\Users\User\Desktop\8vo\Sistemas-Distribuidos\Sistemas-Distribuidos
\Workshop5\Actividad4_Mutual_Exclusion>

PS C:\Users\User\Desktop\8vo\Sistemas-Distribuidos\Sistemas-Distribuidos
\Workshop5\Actividad4_Mutual_Exclusion> "C
PS C:\Users\User\Desktop\8vo\Sistemas-Distribuidos\Sistemas-Distribuidos
\Workshop5\Actividad4_Mutual_Exclusion> python central_server.py client
2
[Cliente-2] En cola...
[Cliente-2] Acceso concedido - entrando a seccion critica
[Cliente-2] Saliendo de seccion critica
[Cliente-2] En cola...
[Cliente-2] Acceso concedido - entrando a seccion critica
[Cliente-2] Saliendo de seccion critica
[Cliente-2] Acceso concedido - entrando a seccion critica
[Cliente-2] Saliendo de seccion critica
PS C:\Users\User\Desktop\8vo\Sistemas-Distribuidos\Sistemas-Distribuidos
\Workshop5\Actividad4_Mutual_Exclusion>

```

Figure 3: Execution output demonstrating mutual exclusion (central server and token ring).

6 Conclusions

- **Part 1:** The centralized UTC server is simple and effective for time synchronization. Adding timeouts and exception handling improves robustness against network issues.
- **Part 2:** Decentralized synchronization through averaging converges toward a global time, but periodic drift disrupts convergence. The algorithm shows that coordination without a central authority is feasible but requires more rounds to stabilize.
- **Part 3:** Vector clocks capture causal ordering without synchronized physical clocks. They can distinguish between causally related and concurrent events, unlike Lamport timestamps, at the cost of $O(n)$ overhead per message.
- **Part 4:** Both mutual exclusion approaches guarantee safety. The central server provides FIFO fairness but has a single point of failure; the token ring is more fault-tolerant but introduces latency and the risk of token loss.

ZeroMQ proved versatile for all four mechanisms, offering appropriate socket patterns for each communication need: REQ/REP for request-reply, PUSH/PULL for unidirectional messaging, and ROUTER/DEALER for multiplexed client-server interaction.

7 References

1. Coulouris, G., Dollimore, J., Kindberg, T., & Blair, G. (2012). *Distributed Systems: Concepts and Design*. 5th Edition. Addison-Wesley.
2. Tanenbaum, A. S., & Van Steen, M. (2017). *Distributed Systems: Principles and Paradigms*. 3rd Edition. Pearson.
3. ZeroMQ Documentation. <https://zeromq.org/>

4. Lamport, L. (1978). Time, Clocks, and the Ordering of Events in a Distributed System. *Communications of the ACM*, 21(7), 558–565.
5. Fidge, C. J. (1988). Timestamps in Message-Passing Systems That Preserve the Partial Ordering. *Proceedings of the 11th Australian Computer Science Conference*.